

Functions Continued

Topics in this Section

1. **Recursion**
 2. **Functions as First-Class Members**
 3. **Function Expressions**
 4. **Synchronous Callbacks**
 5. **Asynchronous Callbacks**
 6. **Arrow Functions**
-

1. Parameters Validation

- Functions can interrupt execution with the return keyword to handle invalid inputs.
- Example: Validate whether a function parameter is an array before processing it.

javascript

Copy code

```
function getMeanTemp(temperatures) {  
  if (!(temperatures instanceof Array)) {  
    return NaN; // Handle invalid input  
  }  
  
  let sum = 0;  
  
  for (let i = 0; i < temperatures.length; i++) {  
    sum += temperatures[i];  
  }  
  
  return sum / temperatures.length;  
}
```

- Usage:

javascript

Copy code

```
console.log(getMeanTemp(10)); // -> NaN
```

```
console.log(getMeanTemp([10, 30])); // -> 20
```

2. Recursion

Definition

- Recursion is a function calling itself with modified arguments.
- Example: Factorial function.

Iterative Approach

javascript

Copy code

```
function factorial(n) {  
    let result = 1;  
    while (n > 1) {  
        result *= n;  
        n--;  
    }  
    return result;  
}
```

- Output: factorial(6); // -> 720

Recursive Approach

- Recursive functions simplify notation but must handle termination conditions carefully.

javascript

Copy code

```
function factorial(n) {  
    return n > 1 ? n * factorial(n - 1) : 1;  
}
```

- Output: factorial(6); // -> 720
-

3. Functions as First-Class Members

- Functions can be treated like any other data type (e.g., assigned to variables or passed as arguments).
- Example:

javascript

Copy code

```
function showMessage(message) {  
    console.log(`Message: ${message}`);  
}  
  
let sm = showMessage; // Assign function to variable  
sm("This works!"); // -> Message: This works!
```

- Passing functions as arguments:

javascript

Copy code

```
function add(a, b) {  
    return a + b;  
}  
  
function operation(func, x, y) {  
    return func(x, y);  
}  
  
console.log(operation(add, 10, 20)); // -> 30
```

4. Function Expressions

Named Function Expression

javascript

Copy code

```
let myAdd = function add(a, b) {  
    return a + b;
```

```
};
```

```
console.log(myAdd(10, 20)); // -> 30
```

Anonymous Function Expression

- Define a function without a name:

javascript

Copy code

```
let myAdd = function(a, b) {
```

```
    return a + b;
```

```
};
```

```
console.log(myAdd(10, 20)); // -> 30
```

- Inline usage with callbacks:

javascript

Copy code

```
function operation(func, x, y) {
```

```
    return func(x, y);
```

```
}
```

```
console.log(operation(function(a, b) {
```

```
    return a * b;
```

```
}, 10, 20)); // -> 200
```

5. Callbacks

Definition

- A function passed as an argument to another function, executed at a specific time.

Synchronous Callbacks

- Execution happens in sequence.

javascript

Copy code

```
let inner = function() {
```

```
    console.log('inner 1');  
};
```

```
let outer = function(callback) {  
    console.log('outer 1');  
    callback();  
    console.log('outer 2');  
};
```

```
console.log('test 1');  
outer(inner);  
console.log('test 2');
```

- Output:

sql

Copy code

test 1

outer 1

inner 1

outer 2

test 2

Asynchronous Callbacks

- Callback execution depends on external events, e.g., timers, user actions, or server responses.
- Example: setTimeout function:

javascript

Copy code

```
setTimeout(() => {  
    console.log('Executed after 1 second');  
}, 1000);
```

6. Arrow Functions

- Concise syntax for writing functions.

javascript

Copy code

```
let add = (a, b) => a + b;
```

```
console.log(add(10, 20)); // -> 30
```

- No binding of this keyword; context remains unchanged.